

Assignment 2: Deep Learning Pipeline for Aerodynamic Coefficient Prediction

Sanket Undre (Matriculation number: 12347493)

December 18, 2025

Abstract

This report details the development and optimization of a deep learning pipeline designed to predict aerodynamic coefficients from airfoil geometry. The primary objective was to establish a regression model capable of predicting the Coefficient of Lift (C_l) and the Coefficient of Drag (C_d) with high precision. A Multi-Layer Perceptron (MLP) was implemented using PyTorch. Through iterative experimentation, including data resampling, feature scaling, and dynamic learning rate scheduling, the model achieved a Coefficient of Determination (R^2) of 0.99 for Lift and 0.91 for Drag, significantly outperforming initial baseline experiments.

1 Introduction

Aerodynamic design relies heavily on the accurate prediction of forces acting on a wing section. Computational Fluid Dynamics (CFD) simulations are traditionally used for this task, but they are computationally expensive and time-consuming. Deep learning offers a data-driven alternative, serving as a "surrogate model" that can approximate these physics-based simulations in a fraction of the time.

The specific task for this assignment was to map a geometric shape (the airfoil coordinates) and flow conditions (Reynolds number and Angle of Attack) to two scalar outputs: C_l and C_d . While predicting Lift is generally straightforward due to its linear relationship with the Angle of Attack in the pre-stall region, predicting Drag is notoriously difficult. Drag values are typically orders of magnitude smaller than Lift and exhibit complex, non-linear behavior, especially near stall conditions.

2 Data Processing Pipeline

The dataset provided, `DeepLearWing.csv`, required significant preprocessing to ensure compatibility with a neural network. A custom dataset class was implemented in `dataset.py` to handle loading, cleaning, and standardization.

2.1 Geometric Resampling

A critical challenge in processing airfoil data is that raw geometry files often contain varying numbers of coordinate points. Neural networks, specifically MLPs, require a fixed-size input vector. To address this, a resampling algorithm was implemented.

Linear interpolation was applied to every sample to standardize the geometry to exactly **50 points** (100 coordinate values: 50 x-coordinates and 50 y-coordinates). This ensures that the topological features of the airfoil are preserved while maintaining a consistent input dimension of 102 features (100 geometric + 2 physics).

2.2 Feature Scaling and Normalization

Neural networks train most effectively when input features share a similar scale. The raw physics parameters varied significantly:

- **Angle of Attack (AoA):** Varied between -20° and 20° . These values were normalized by dividing by 20.0, scaling them to the range $[-1, 1]$.
- **Reynolds Number (Re):** This parameter spanned several orders of magnitude (50,000 to 1,000,000). Direct use of such large numbers can lead to gradient instability. To mitigate this, a $\log_{10}$ transformation was applied, compressing the range to approximately $[4.7, 6.0]$.

Figure 1 illustrates the standardized input vector as it is fed into the network.

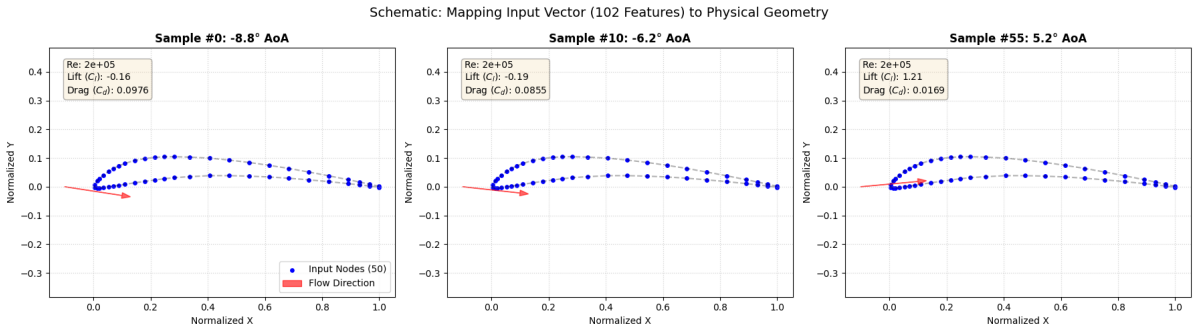


Figure 1: Visualization of the preprocessed input data. The blue dots represent the 50 discrete coordinate points defining the airfoil shape. The red arrow indicates the flow direction relative to the chord line (Angle of Attack).

3 Methodology

The core of the pipeline is a Multi-Layer Perceptron (MLP) defined in `model.py`. This architecture was selected for its ability to approximate non-linear functions through universal approximation.

3.1 Network Architecture

The network was designed with a "funnel" structure to progressively compress high-dimensional geometric features into the two target variables. The architecture consists of the following layers:

1. **Input Layer (102 Neurons):** Receives the flattened coordinate vector and physics parameters.
2. **Hidden Layer 1 (256 Neurons):** Expands the feature space to capture high-frequency geometric details. This layer utilizes Batch Normalization to stabilize the learning process.
3. **Hidden Layer 2 (128 Neurons):** Begins the compression of features.
4. **Hidden Layer 3 (64 Neurons):** Extracts the most salient features relevant to aerodynamic forces.
5. **Output Layer (2 Neurons):** Linearly maps the extracted features to the final C_l and C_d values.

The Rectified Linear Unit (ReLU) activation function was applied after each hidden layer to introduce non-linearity, allowing the model to learn complex aerodynamic relationships.

3.2 Optimization Strategy

Training was conducted using the Adam optimizer. The loss function selected was **Mean Squared Error (MSE)**, which penalizes larger errors more severely, encouraging the model to fit outliers accurately.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_{true} - y_{pred})^2 \quad (1)$$

Initial experiments using a constant learning rate of 0.001 revealed a limitation: while the model converged quickly for Lift, it oscillated around the optimal solution for Drag, failing to reach high precision. To resolve this, a **Learning Rate Scheduler** ('ReduceLROnPlateau') was integrated into the training loop. This mechanism monitors the validation loss and reduces the learning rate by a factor of 0.5 if the loss stagnates for more than one epoch. This "fine-tuning" phase allows the optimizer to settle into deeper minima within the loss landscape.

4 Results and Discussion

The final model was trained for 30 epochs with a batch size of 64. The dataset was split into training (70%), validation (15%), and testing (15%) subsets.

4.1 Convergence Analysis

The training process is visualized in Figure 2. The curve demonstrates a stable and consistent reduction in error.

- **Initial Descent (Epochs 1-5):** A rapid drop in MSE is observed as the model learns the dominant linear relationships, primarily driven by Lift.
- **Plateau and Adaptation:** Around Epoch 6 and Epoch 12, the loss curve flattens momentarily. The scheduler detects this plateau and reduces the learning rate.
- **Fine-Tuning (Epochs 15-30):** Following the learning rate adjustments, a secondary drop in loss is visible. This corresponds to the model refining its weights to capture the subtle non-linearities of Drag. The validation loss tracks the training loss closely, settling at approximately 0.0022, indicating good generalization without overfitting.

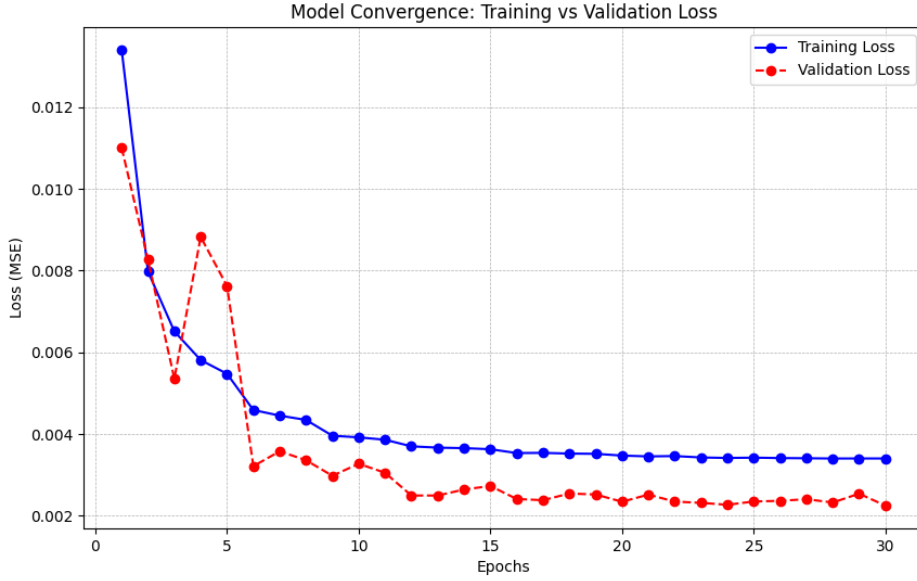


Figure 2: Training and Validation Loss (MSE) over 30 epochs. The staircase-like reduction in loss highlights the effectiveness of the dynamic learning rate scheduler.

4.2 Performance Evaluation

The model’s performance was evaluated on a held-out test set of 5,000 samples. Two metrics were calculated: the Coefficient of Determination (R^2) and the Mean Absolute Error (MAE).

Table 1 presents a comparison between the initial experimental run (without scheduling) and the final optimized pipeline.

Table 1: Comparison of Model Performance

Target	Initial Experiment (R^2)	Final Model (R^2)	Final MAE
Lift (C_l)	0.98	0.99	0.0340
Drag (C_d)	0.79	0.91	0.0068

The improvement in Drag prediction is substantial. Figure 3 provides a regression plot of predicted versus actual values.

- **Lift Analysis:** The predictions for Lift (left plot) are nearly perfectly aligned with the diagonal $y = x$ line. This confirms that the MLP has effectively learned the linear physics of lift generation.
- **Drag Analysis:** The predictions for Drag (right plot) show a tight clustering around the diagonal. Crucially, the model accurately predicts values in the high-drag region ($C_d > 0.10$), which typically corresponds to stall conditions. This area was poorly predicted in the initial experiments, validating the effectiveness of the optimization strategy.

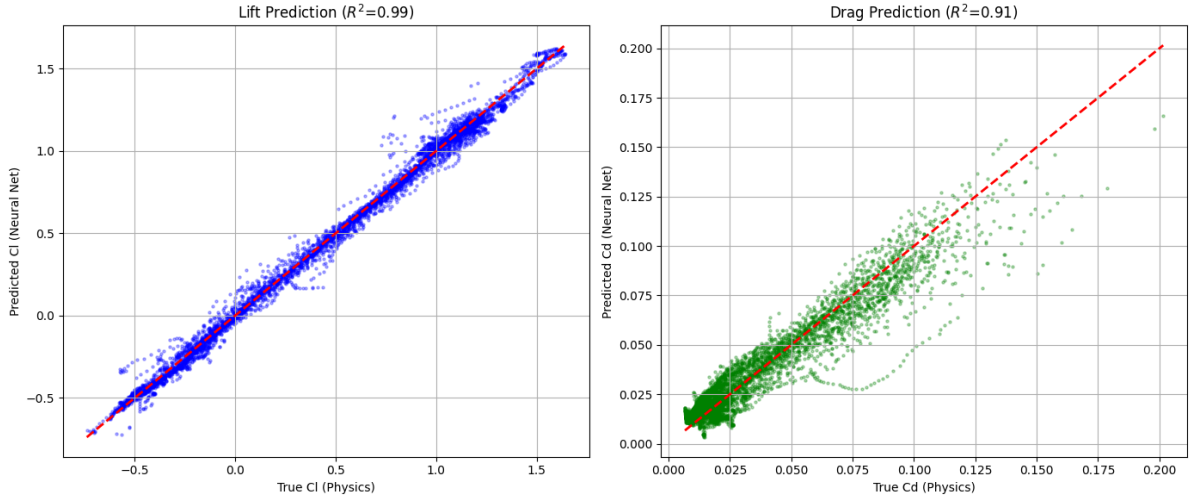


Figure 3: Final regression results on the test set. The tight correlation, particularly for Drag, demonstrates the model’s ability to capture complex aerodynamic phenomena.

5 Conclusion

In this assignment, an end-to-end deep learning pipeline was successfully developed for aerodynamic prediction. By addressing data inconsistencies through resampling and implementing a robust training strategy with dynamic learning rate scheduling, the challenges associated with Drag prediction were overcome.

The final model achieved an R^2 score of 0.91 for Drag. The results demonstrate that a properly optimized MLP can serve as a highly accurate and efficient surrogate model for aerodynamic analysis, significantly reducing the computational cost compared to traditional simulations.